

GIT & GITHUB

GLY606 Water Data Analysis & Modeling

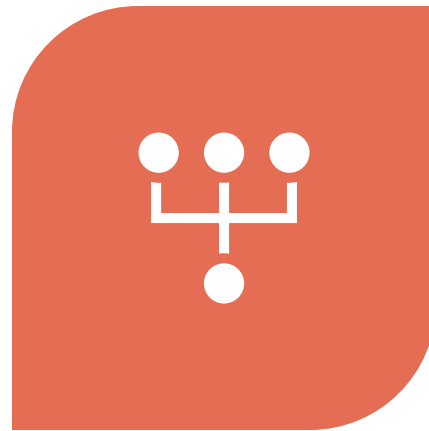
Nov 1st 2024



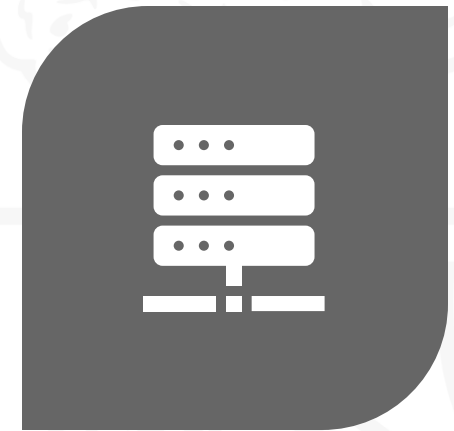
Three use cases



#1: HISTORY OF
VERSIONS



#2: BRANCHING



#3: COLLABORATION
(GITHUB)

Use case #1: history of versions

- Basic workflow
- **Adding** and **committing** files
- The git log
- The **staging** area
- **Removing** files
- Viewing diffs of files
- The .gitignore file



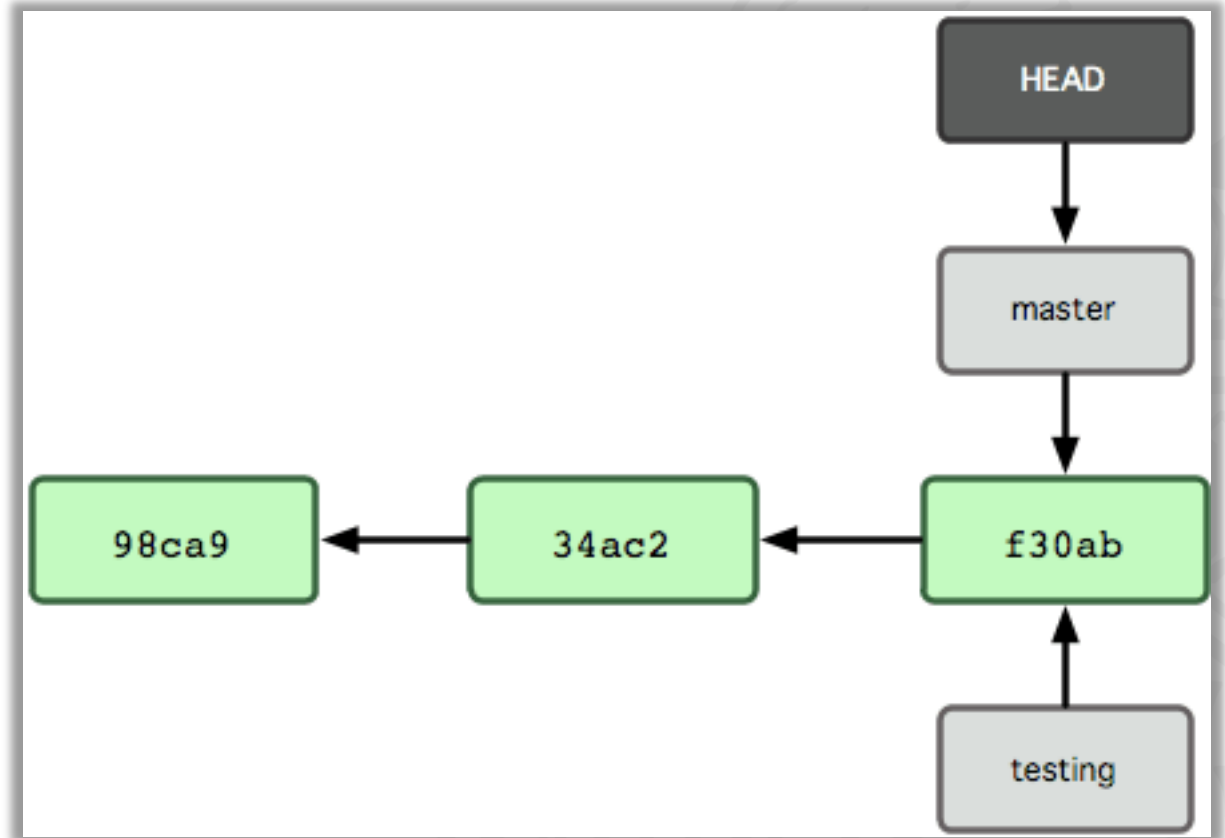
Use case #2: branching

- What is a branch?
- **Branching** commands
- The HEAD pointer
- Basics of **merging**
- Basics of **rebasing**
- Aside: the git reset command

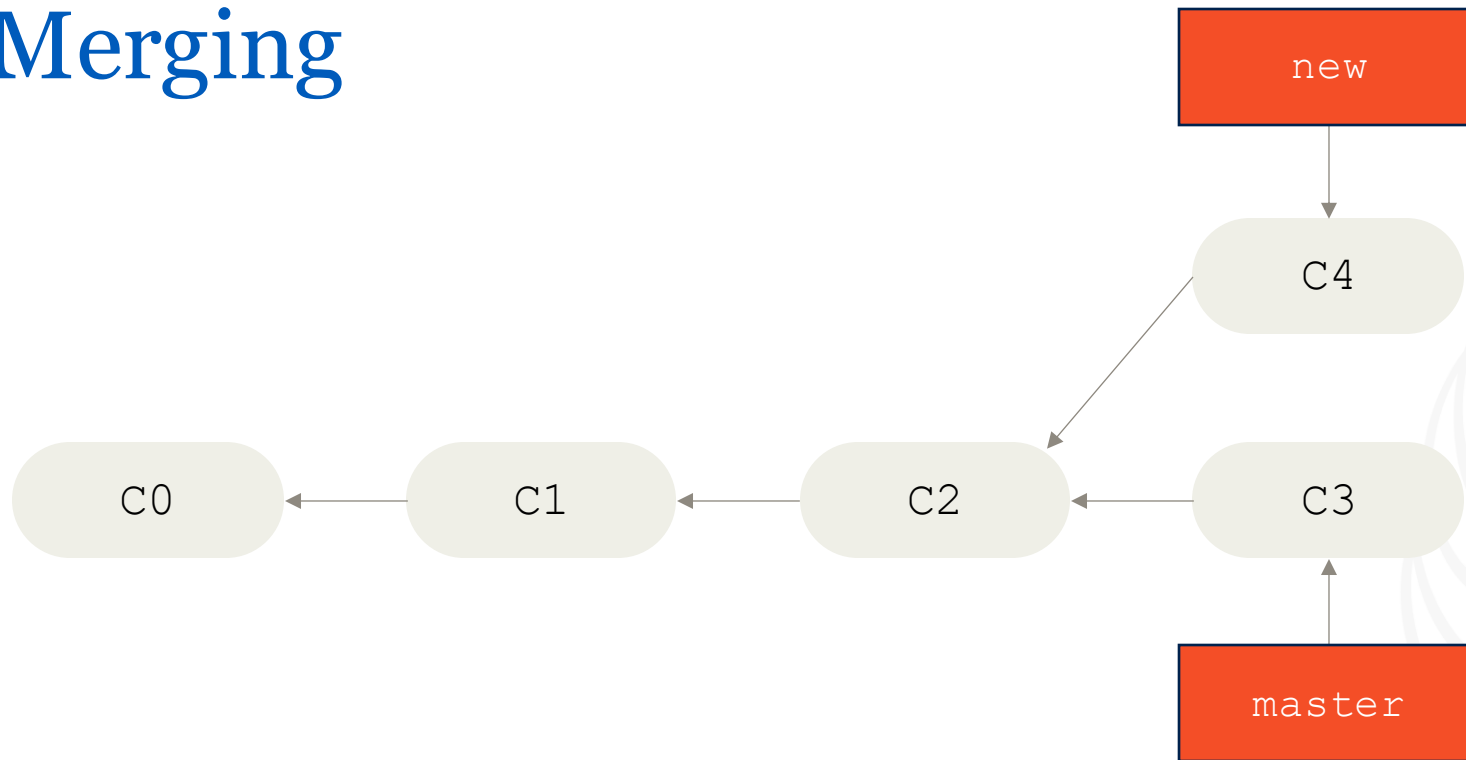


The HEAD pointer

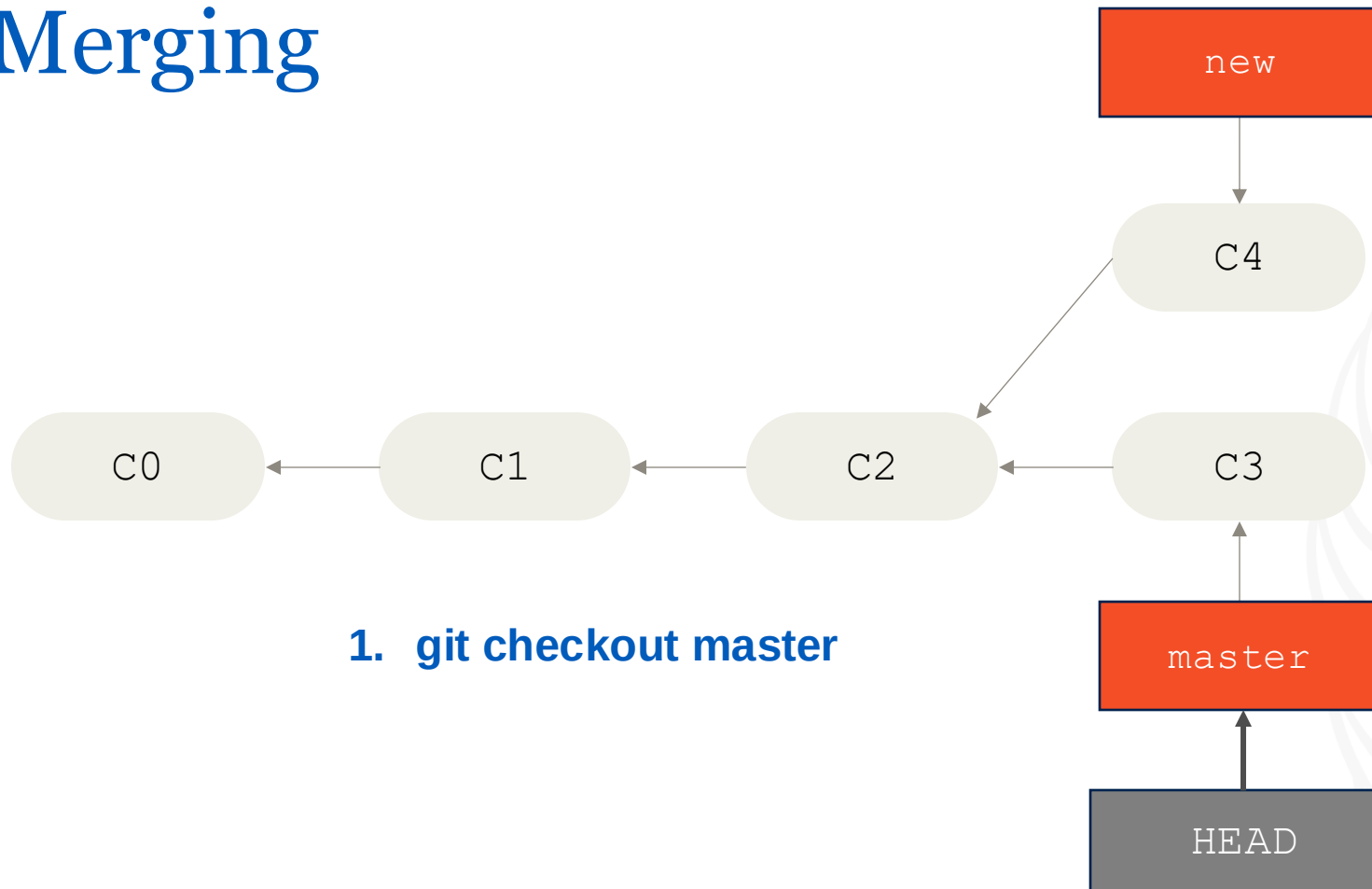
- Recall: all branches simply a pointer to a commit
- HEAD: a special point to the current branch, moves around as you switch branches



Merging

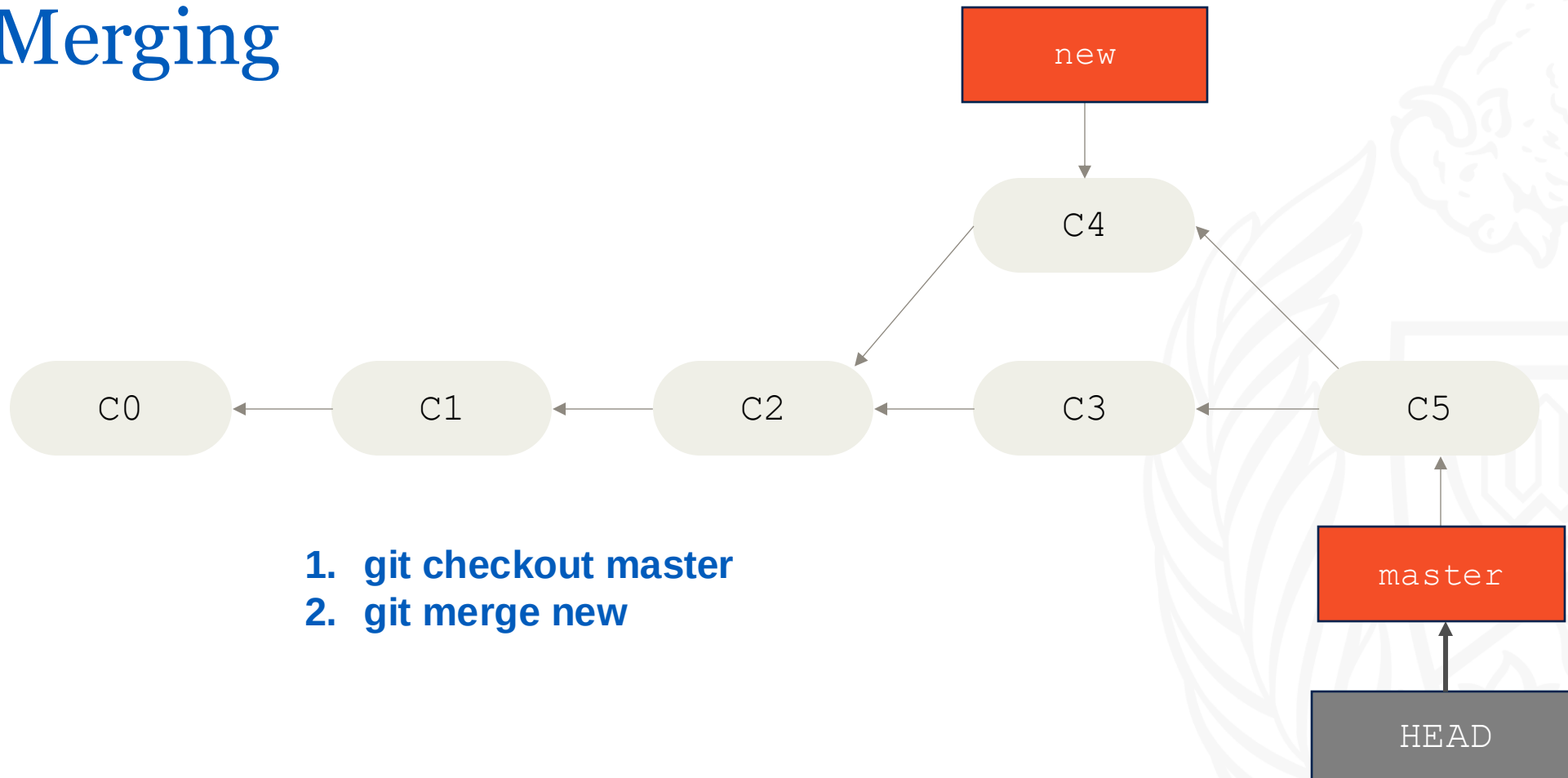


Merging



1. git checkout master

Merging



Merge conflicts

- Before merge... master and new have diverged (**commit 4** on master vs. **commit 1** on new branch)

```

(base) jovyan@jupyter-yifancheng-5fub:~/test/git_practice$ git branch
  master
* new
(base) jovyan@jupyter-yifancheng-5fub:~/test/git_practice$ git log
commit 20052357416b3e1400f64c4041656e45c3897bcd (HEAD -> new)

Author: Yifan Cheng <yifanc6@uw.edu>
Date:   Wed Oct 30 15:09:46 2024 +0000

    commit 1 on new branch

commit 45fd1351247988866cacf576ec0d14f0afe69fe7
Author: Yifan Cheng <yifanc6@uw.edu>
Date:   Wed Oct 30 15:07:56 2024 +0000

    add file "b"

commit e9c053c5748f45edae5c434cea8cf594b68930e5
Author: Yifan Cheng <yifanc6@uw.edu>
Date:   Wed Oct 30 13:54:39 2024 +0000

    commit unstaged changes using "git commit -a"

commit d5bcff0736e24f409600ffafb05a52ed1ca0bc7c
Author: Yifan Cheng <yifanc6@uw.edu>
Date:   Wed Oct 30 13:52:30 2024 +0000

    add file with name "a"
    
```

```

(base) jovyan@jupyter-yifancheng-5fub:~/test/git_practice$ git branch
* master
  new
(base) jovyan@jupyter-yifancheng-5fub:~/test/git_practice$ git log
commit b50e2ca3532e5da63764c293643d4d50e86aea8e (HEAD -> mast
er)

Author: Yifan Cheng <yifanc6@uw.edu>
Date:   Wed Oct 30 15:11:10 2024 +0000

    commit 4 on master

commit 45fd1351247988866cacf576ec0d14f0afe69fe7
Author: Yifan Cheng <yifanc6@uw.edu>
Date:   Wed Oct 30 15:07:56 2024 +0000

    add file "b"

commit e9c053c5748f45edae5c434cea8cf594b68930e5
Author: Yifan Cheng <yifanc6@uw.edu>
Date:   Wed Oct 30 13:54:39 2024 +0000

    commit unstaged changes using "git commit -a"

commit d5bcff0736e24f409600ffafb05a52ed1ca0bc7c
Author: Yifan Cheng <yifanc6@uw.edu>
Date:   Wed Oct 30 13:52:30 2024 +0000

    add file with name "a"
    
```

Merging conflicts

- Sometimes, two branches will edit the same piece of code in different ways.
- Must resolve the conflict manually, then add the conflicting files and explicitly commit.

```

(base) jovyan@jupyter-yifancheng-5fub:~/test/git_practice$ git checkout master
Switched to branch 'master'
(base) jovyan@jupyter-yifancheng-5fub:~/test/git_practice$ git merge new
Auto-merging b
CONFLICT (content): Merge conflict in b
Automatic merge failed; fix conflicts and then commit the result.
    
```

```

$ git checkout master
$ git merge new
$ vim b
    
```

\$ _ jovyan@jupyter-yifancheng X

```

1 This is another test??
2 <<<<<< HEAD
3 Yifan loves strawberry ice cream
4 =====
5 Yifan loves chocolate ice cream
6 >>>>>> new
    
```

Conflict markers

\$ _ jovyan@jupyter-yifancheng X

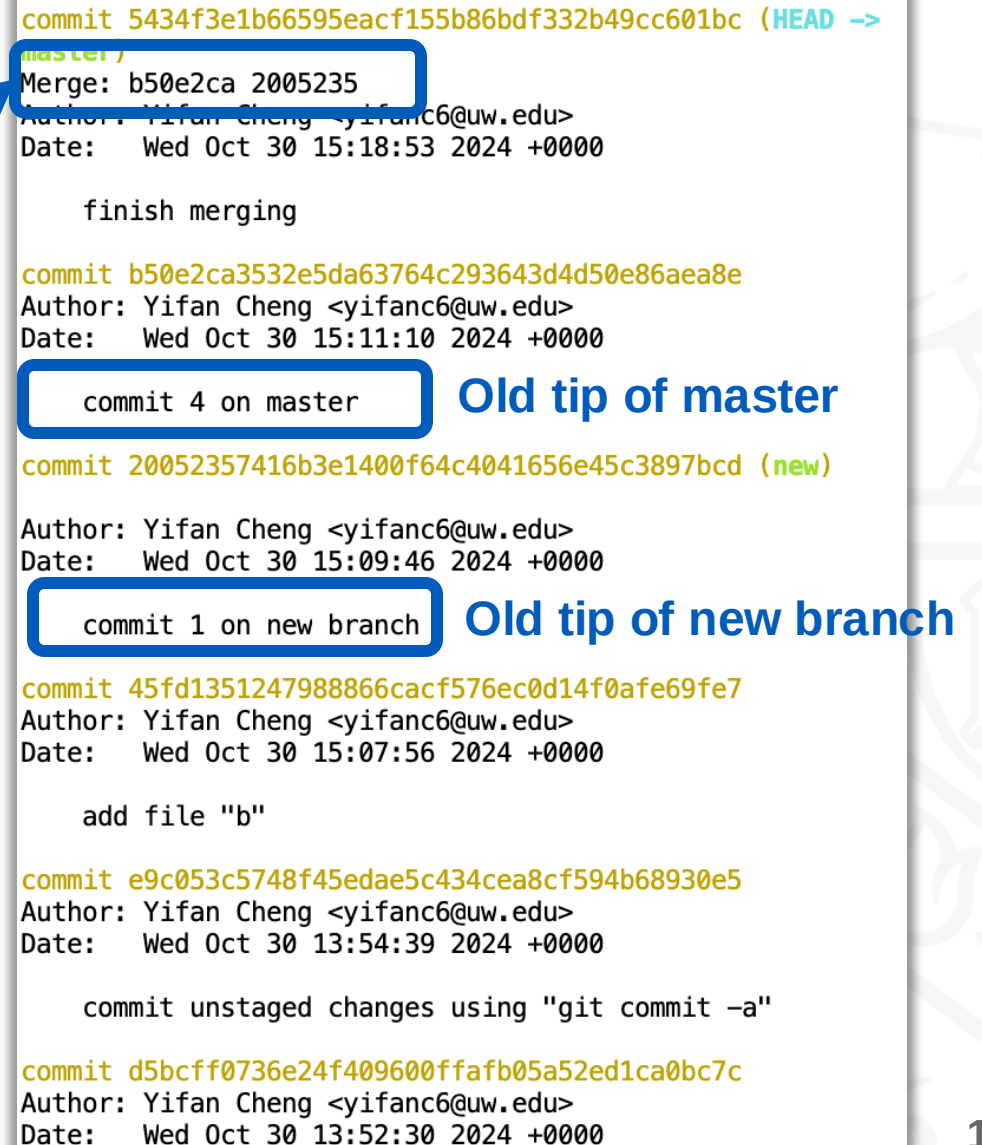
```

1 This is another test??
2 Yifan loves strawberry ice cream
    
```

After we fixed the conflicts in files

- Method 1
 - Step 1: **git add b**
 - Step 2: **git commit -m "finish merging"**
- Method 2
 - **git commit -a**

More on Git Branch & Merge: <https://git-scm.com/book/en/v2/Git-Branching-Basic-Branching-and-Merging>



```
commit 5434f3e1b66595eacf155b86bdf332b49cc601bc (HEAD ->
master)
Merge: b50e2ca 2005235
Author: Yifan Cheng <yifanc6@uw.edu>
Date: Wed Oct 30 15:18:53 2024 +0000

    finish merging

commit b50e2ca3532e5da63764c293643d4d50e86aea8e
Author: Yifan Cheng <yifanc6@uw.edu>
Date: Wed Oct 30 15:11:10 2024 +0000

    commit 4 on master

commit 20052357416b3e1400f64c4041656e45c3897bcd (new)
Author: Yifan Cheng <yifanc6@uw.edu>
Date: Wed Oct 30 15:09:46 2024 +0000

    commit 1 on new branch

commit 45fd1351247988866cacf576ec0d14f0afe69fe7
Author: Yifan Cheng <yifanc6@uw.edu>
Date: Wed Oct 30 15:07:56 2024 +0000

    add file "b"

commit e9c053c5748f45edae5c434cea8cf594b68930e5
Author: Yifan Cheng <yifanc6@uw.edu>
Date: Wed Oct 30 13:54:39 2024 +0000

    commit unstaged changes using "git commit -a"

commit d5bcff0736e24f409600ffafb05a52ed1ca0bc7c
Author: Yifan Cheng <yifanc6@uw.edu>
Date: Wed Oct 30 13:52:30 2024 +0000
```

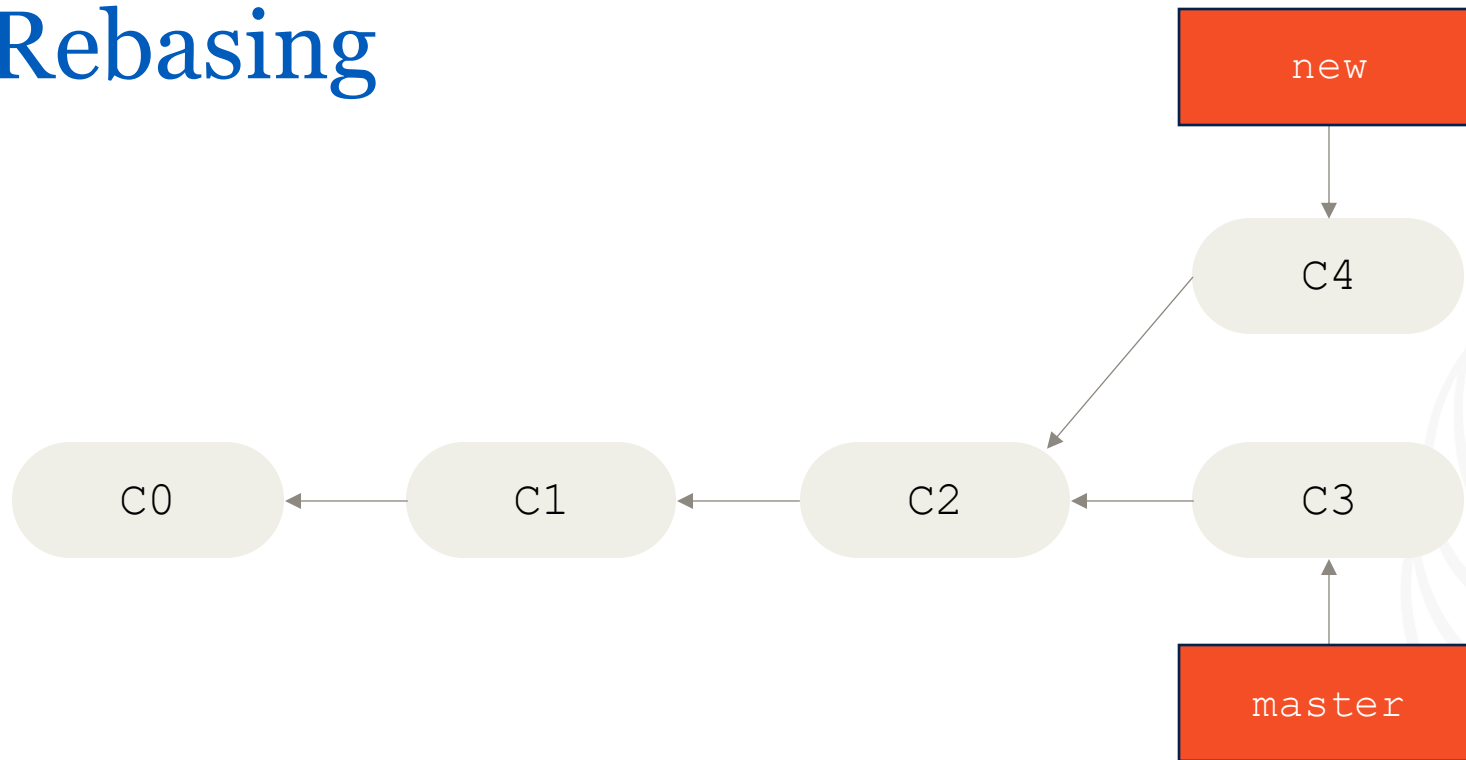
Old tip of master

Old tip of new branch

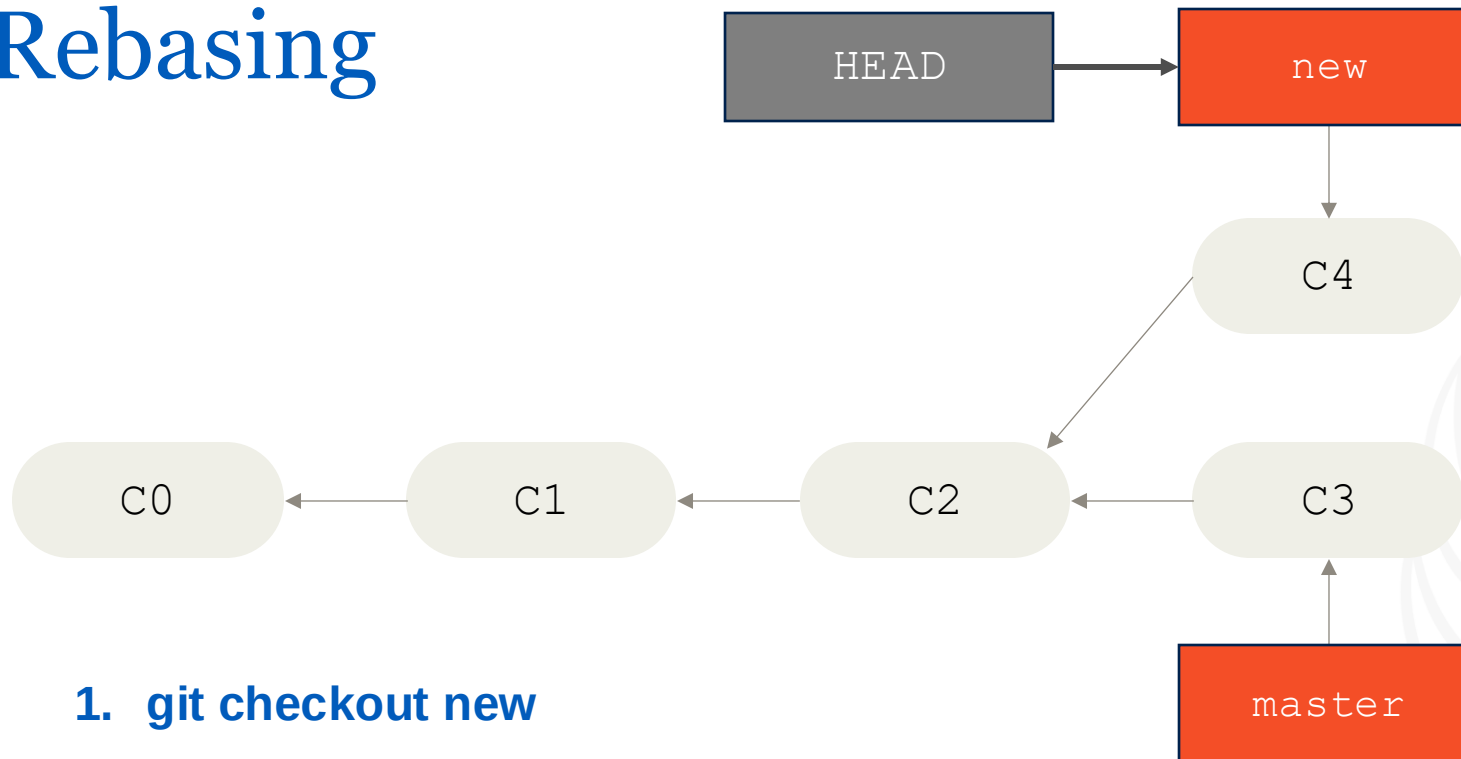
Rebasing

- Instead of a merge, which creates a **new commit** originating from both branches, a **rebase** takes the contents of one branch after the “split” and moves them to the end of the other branch.
- The command **git rebase <basebranch>** takes your currently checked out branch and **replays the diffs** on top of **basebranch**.

Rebasing

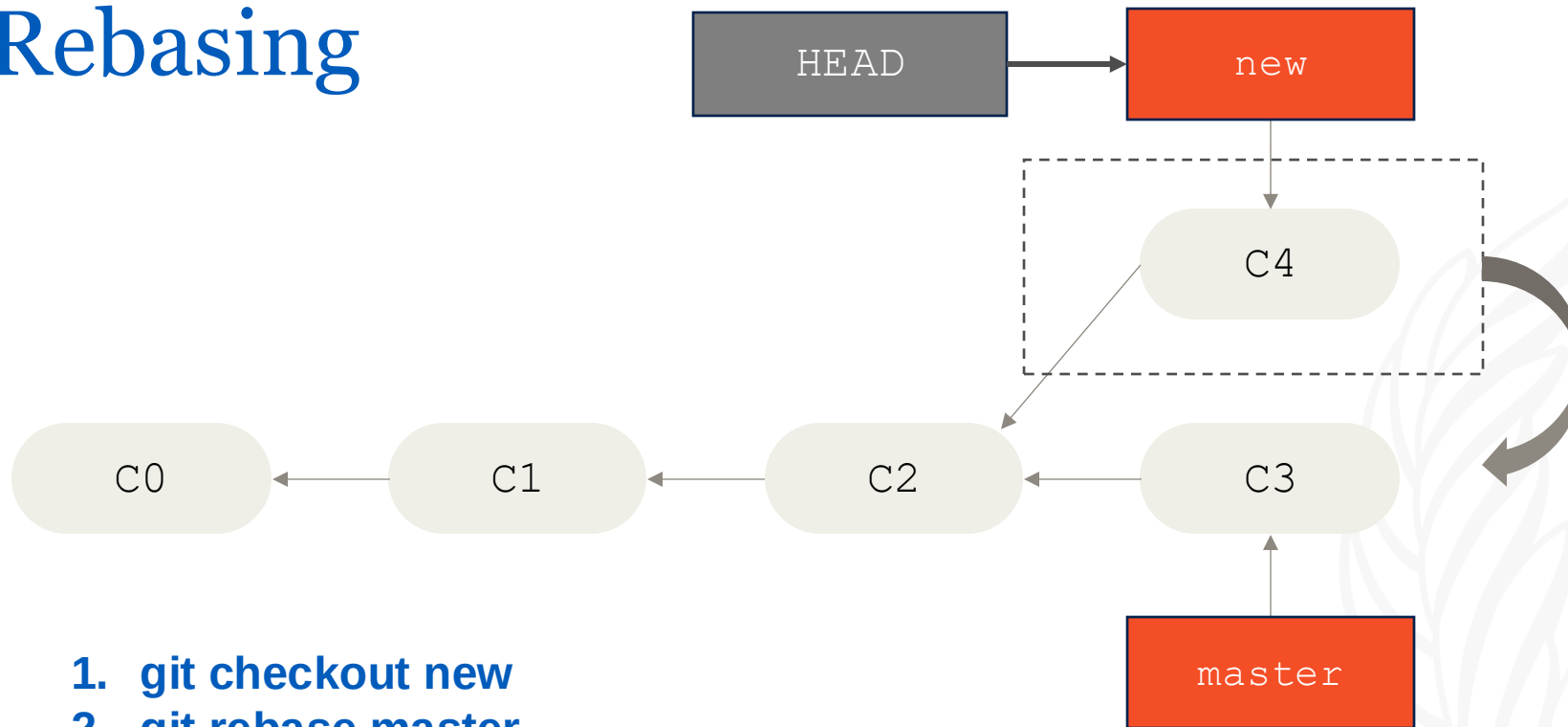


Rebasing



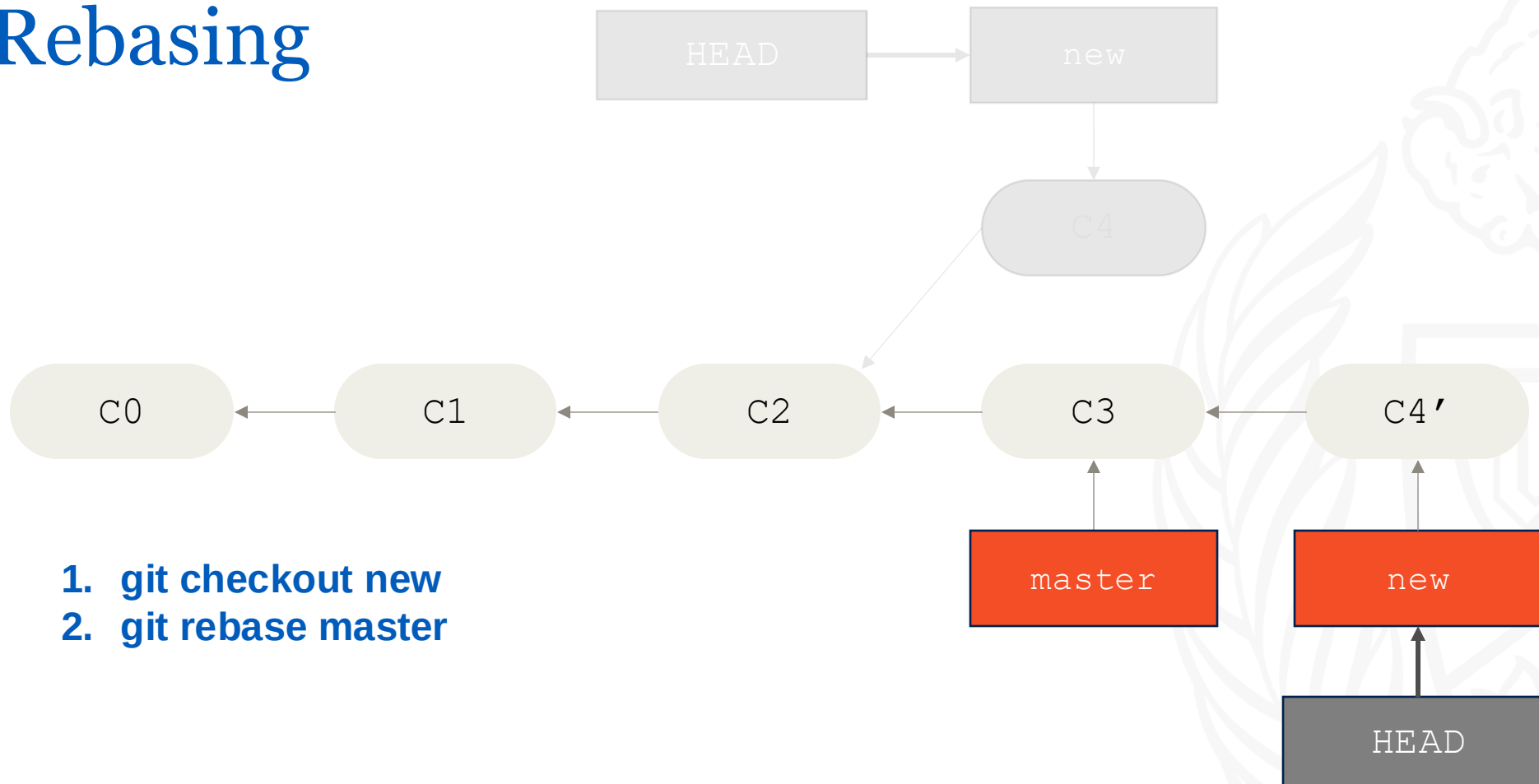
1. `git checkout new`

Rebasing



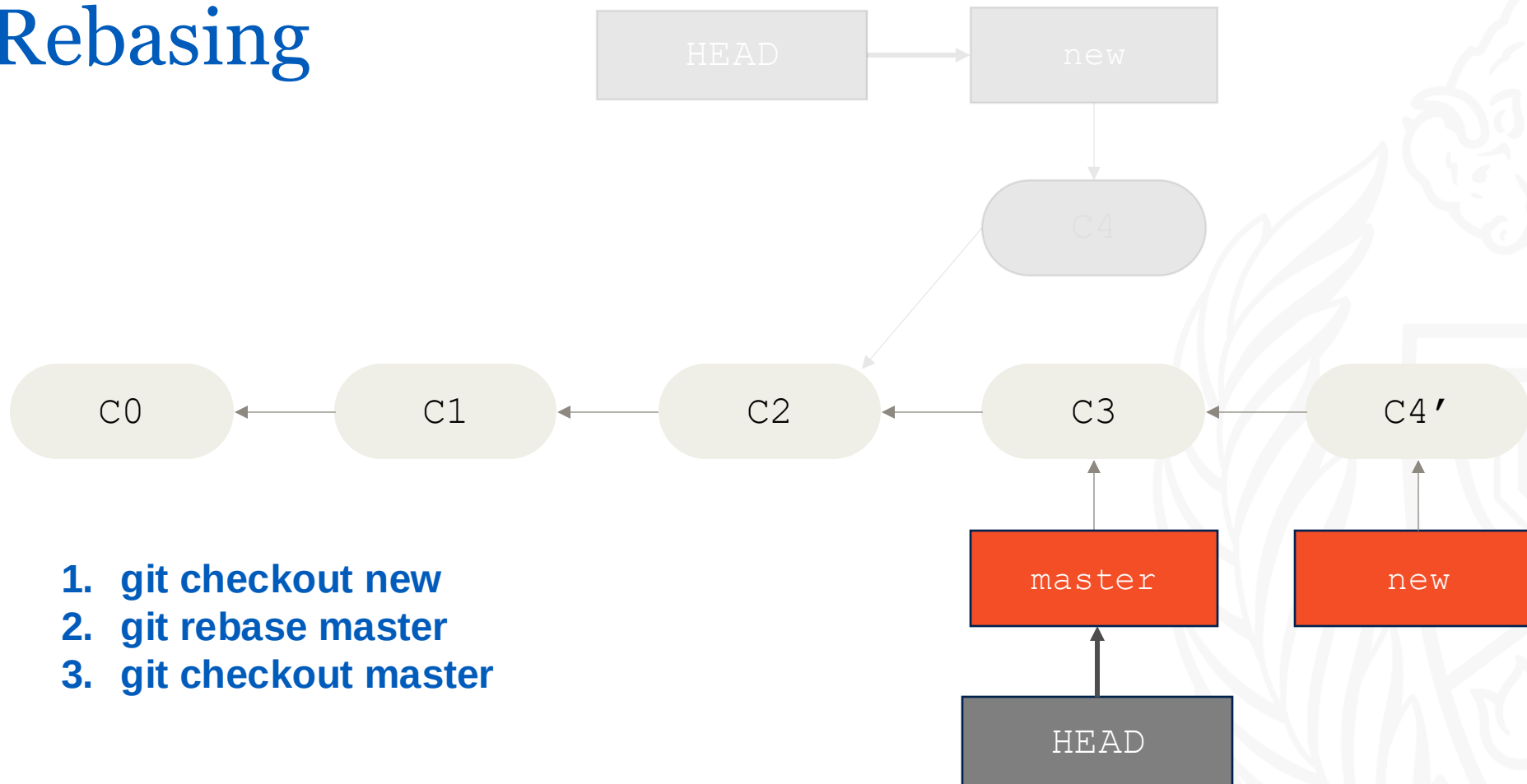
1. **git checkout new**
2. **git rebase master**

Rebasing



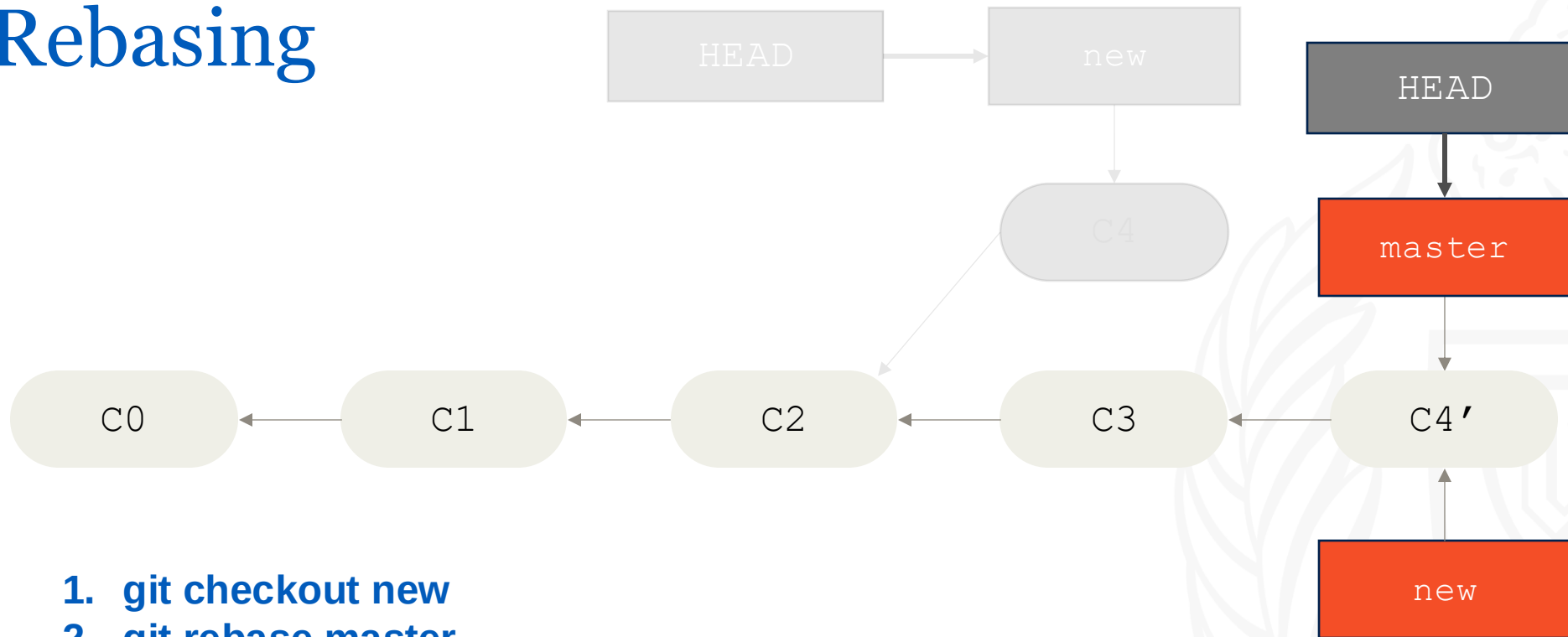
1. **git checkout new**
2. **git rebase master**

Rebasing



1. **git checkout new**
2. **git rebase master**
3. **git checkout master**

Rebasing



1. **git checkout new**
2. **git rebase master**
3. **git checkout master**
4. **git merge new**

Why rebase?

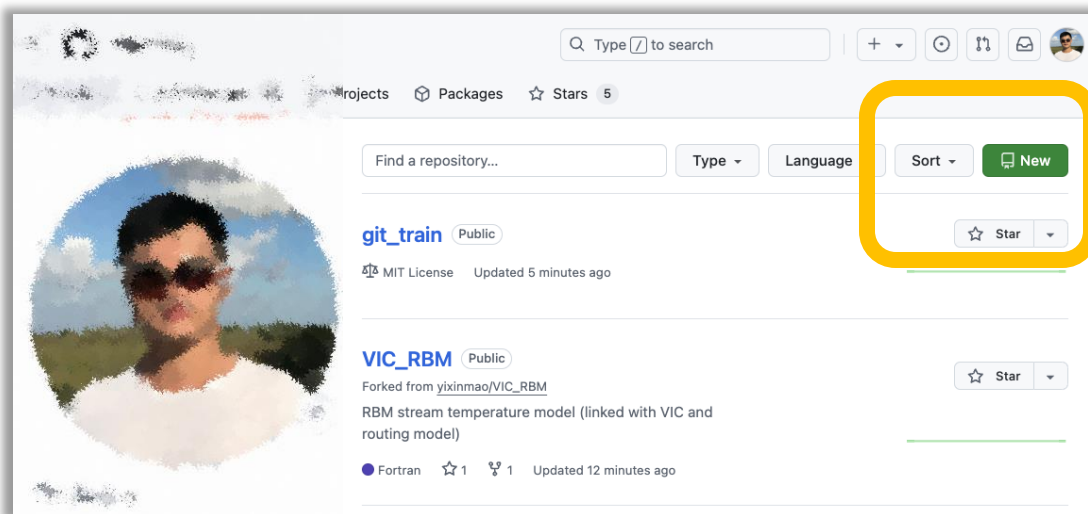
- Creates a linear history; often **cleaner and easier to read**.
- But...**DO. NOT. EVER.** rebase anything that has already been pushed to a repo someone else has access to
 - **Rebasing removes commits** and writes new ones; but someone else might have already based their work off your old commits!
- Resolving conflicts in rebasing is similar to that in “merging”

Use case #3: collaboration

- Creating a **repo** on Github
- **Git clone**
- Remotes
- Remote-tracking branches
- Collaboration workflow



Create a Github Repo



Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

Repository template

No template ▾

Start your repository with a template repository's contents.

Owner *

YifanCheng ▾

git_train_YC

git_train_<initial>

Great repository names are short and memorable. Need inspiration? How about [shiny-couscous](#) ?

Description (optional)

☒ Public

Anyone on the internet can see this repository. You choose who can commit.

☐ Private

You choose who can see and commit to this repository.

Initialize this repository with:

☒ Add a README file

Write a long description for your project. [Learn more about READMEs](#).

Add .gitignore

.gitignore template: None ▾

Choose which files not to track from a list of templates. [Learn more about ignoring files](#).

Choose a license

License: GNU General Public License v3.0 ▾

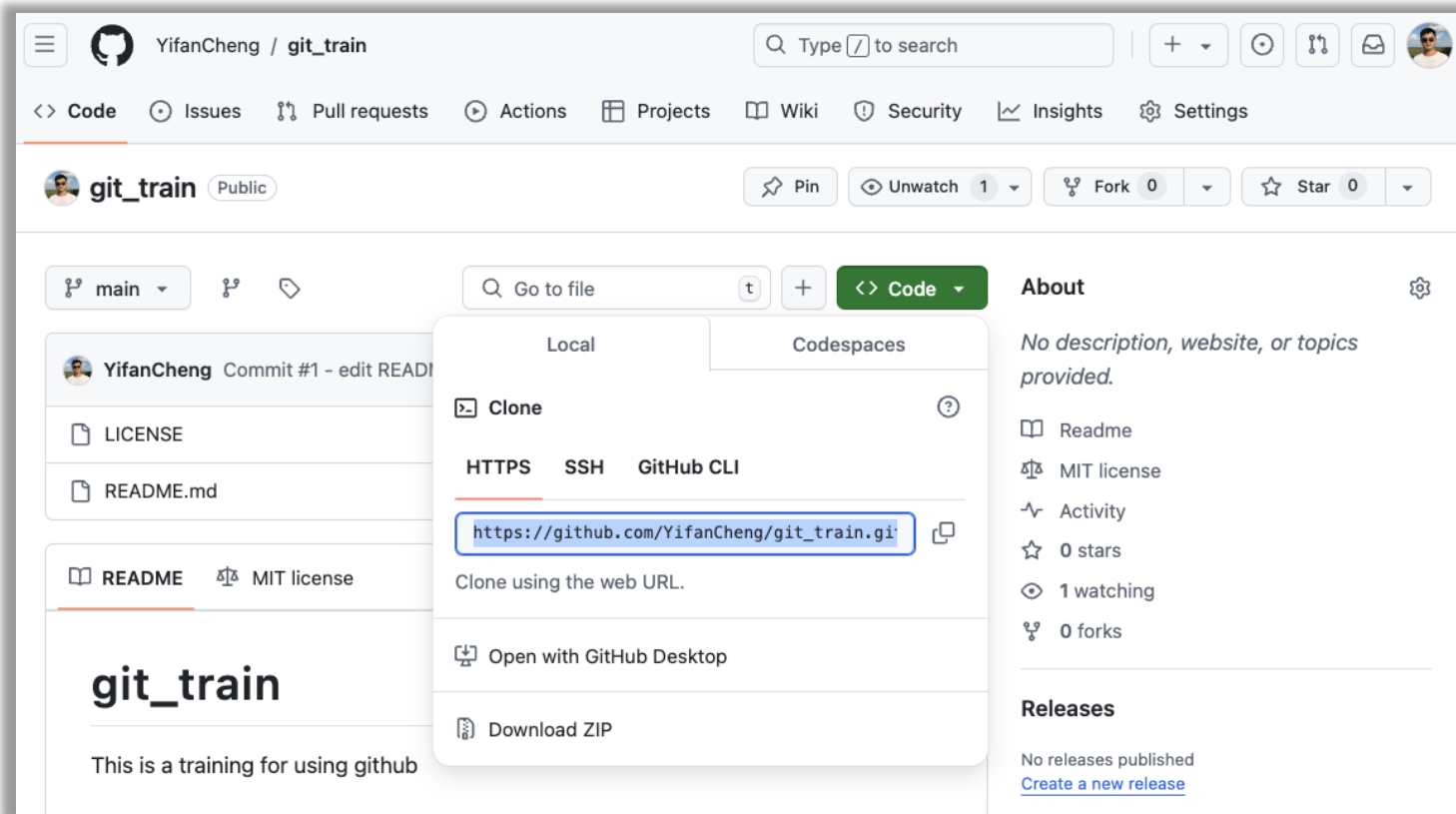
A license tells others what they can and can't do with your code. [Learn more about licenses](#).

This will set `main` as the default branch. Change the default name in your [settings](#).

i You are creating a public repository in your personal account.

Create repository

Git clone



```
$ cd ~  
$ git clone  
https://github.com/<github us  
er name>/git_train <initial>.  
git  
$ cd git_train_<initial>
```

Remotes

- A target computer that has Git repos that you can access
- Via http(s), ssh, or git protocols
- **git remote add <remotename> <remoteaddress>**
- **git remote -v** (view remotes)
- **git remote rm <remotename>**
- Often, with one remote, we name it “origin”

```
$ git remote -v  
$ vim README.md
```

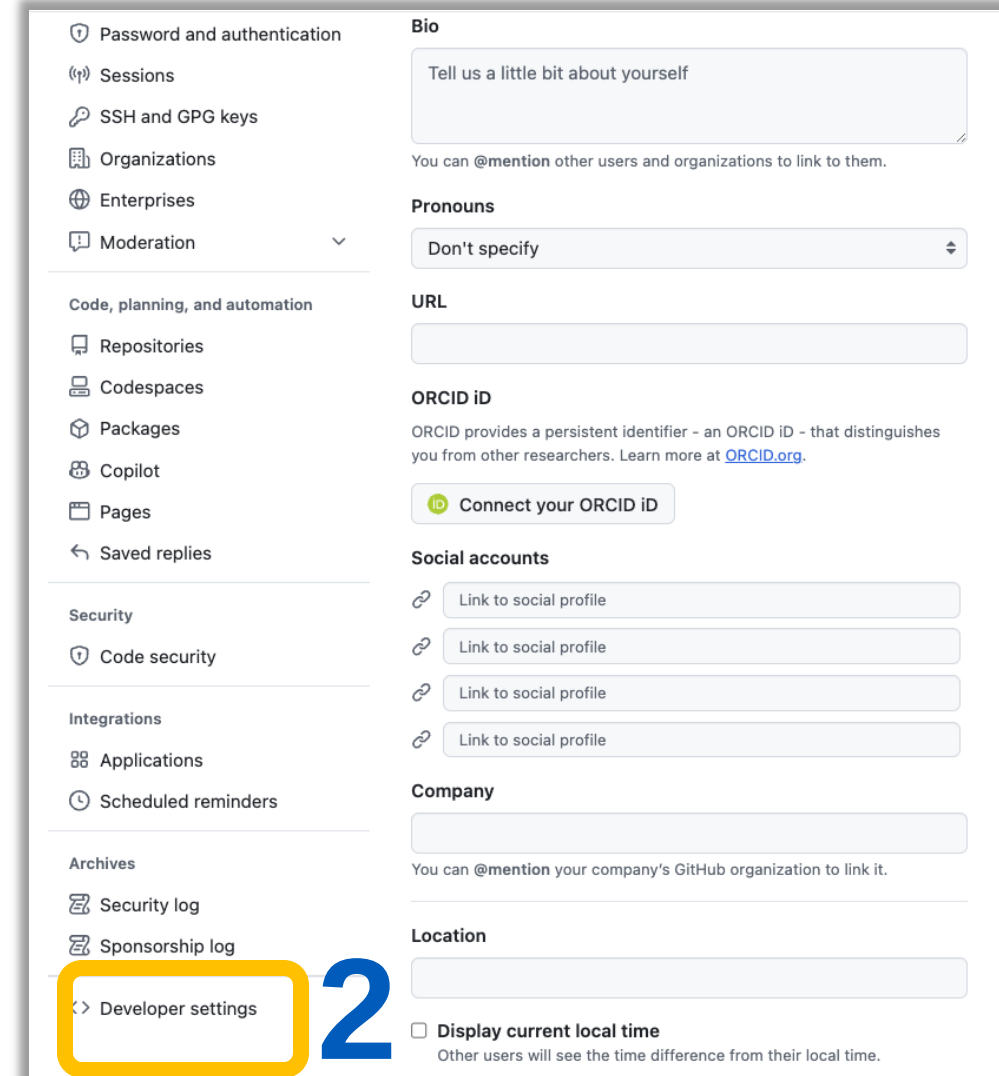
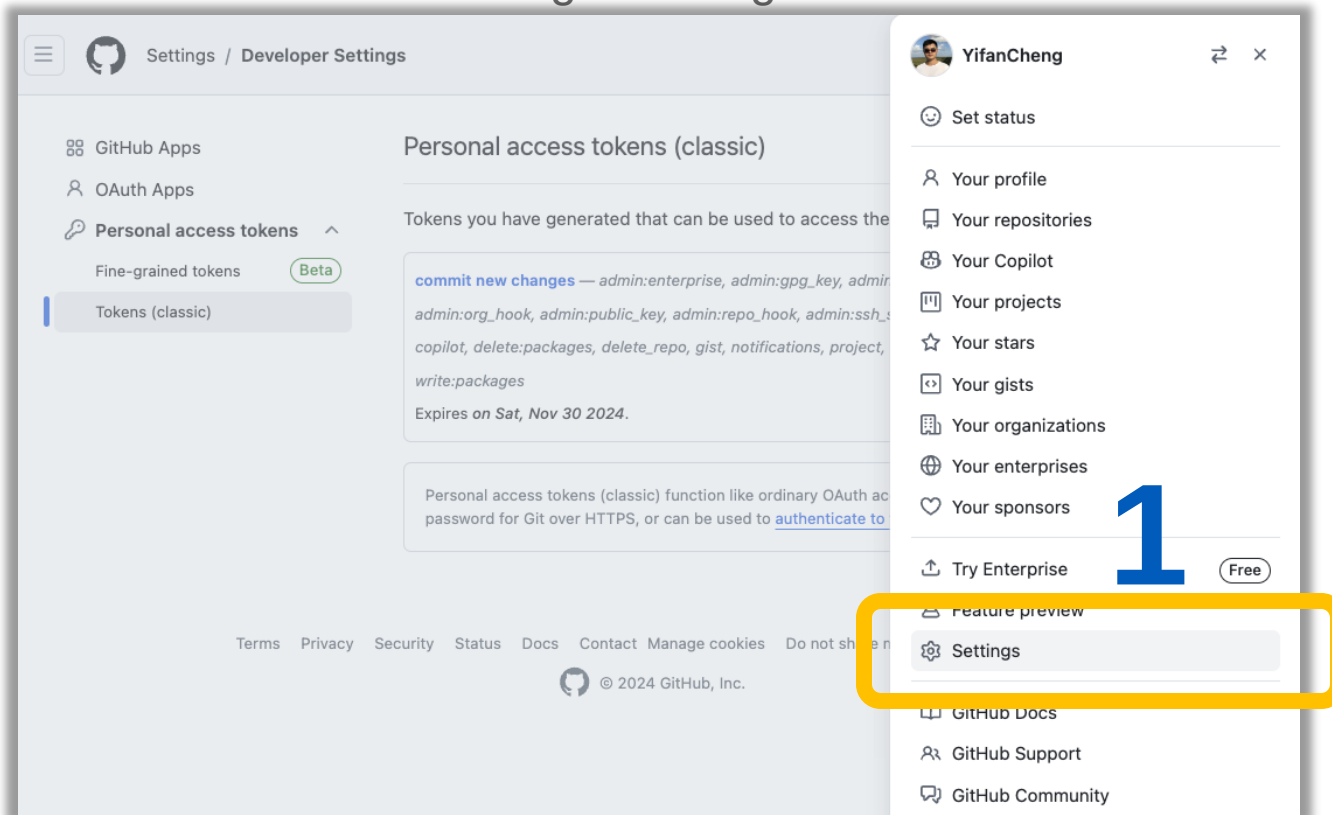
This is a training for using github

```
$ git commit -a
```

Commit #1 – update README

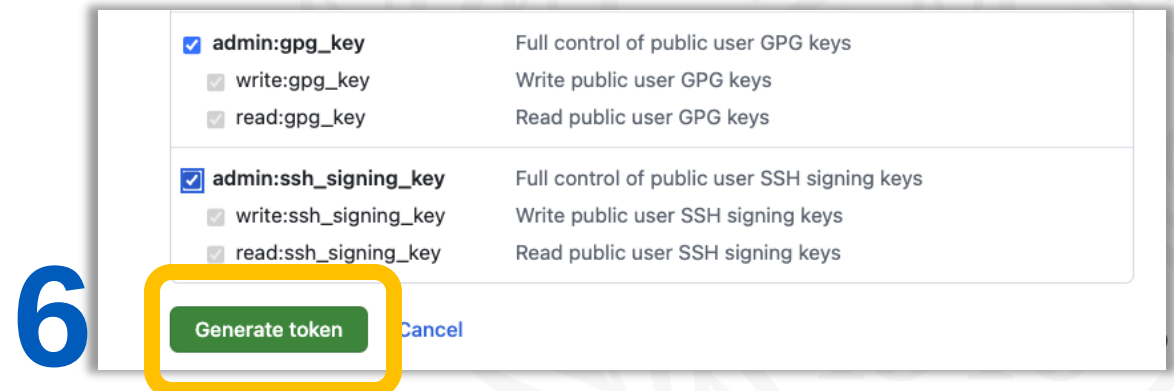
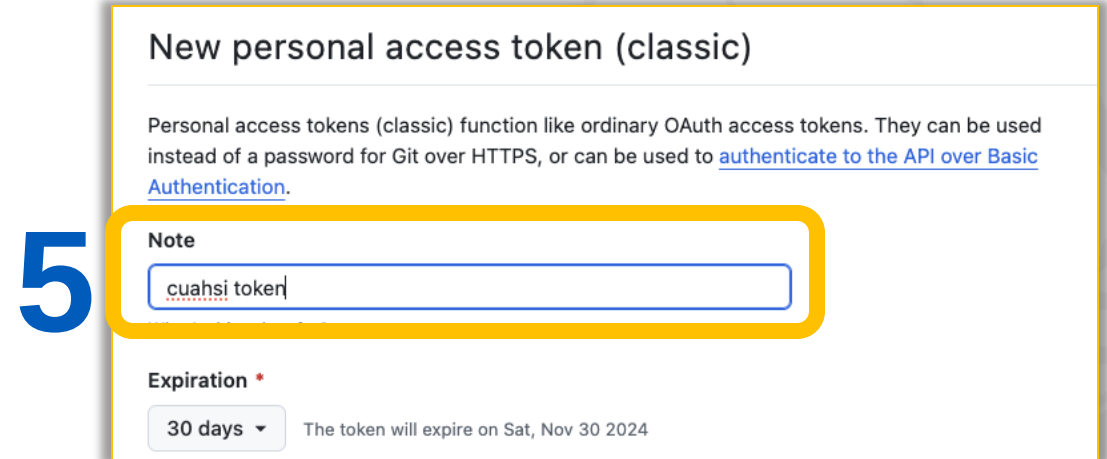
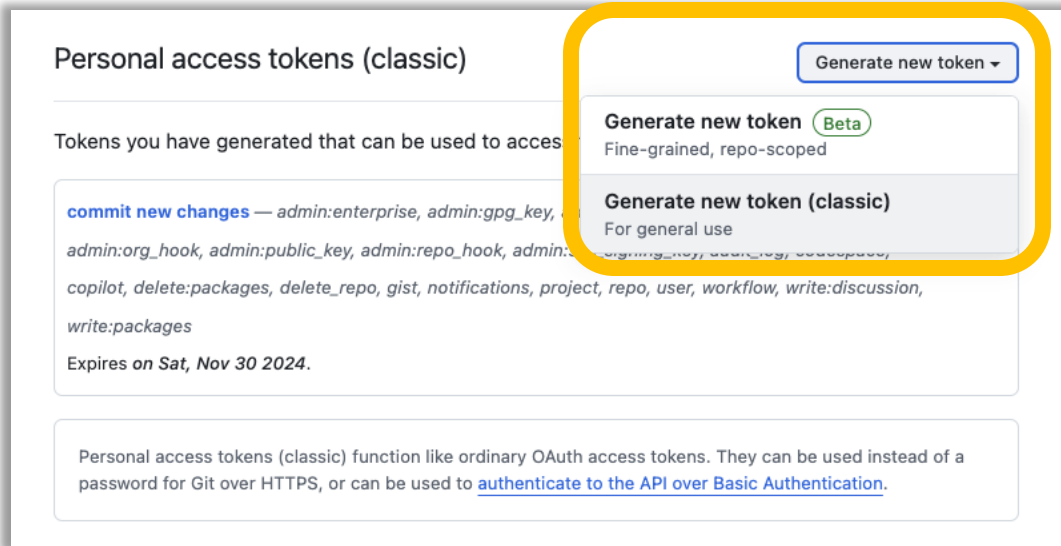
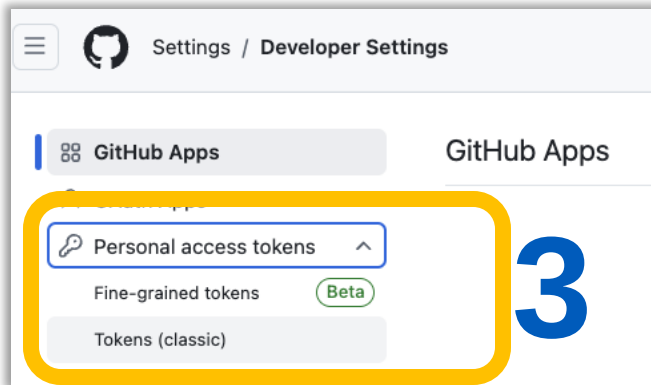
Authenticating to Github

- Here we recommend generating a Personal access token!



Authenticating to Github

After finishing Step 6, copy the token and save it somewhere safe! You will no longer have access to the token once you refresh this page.



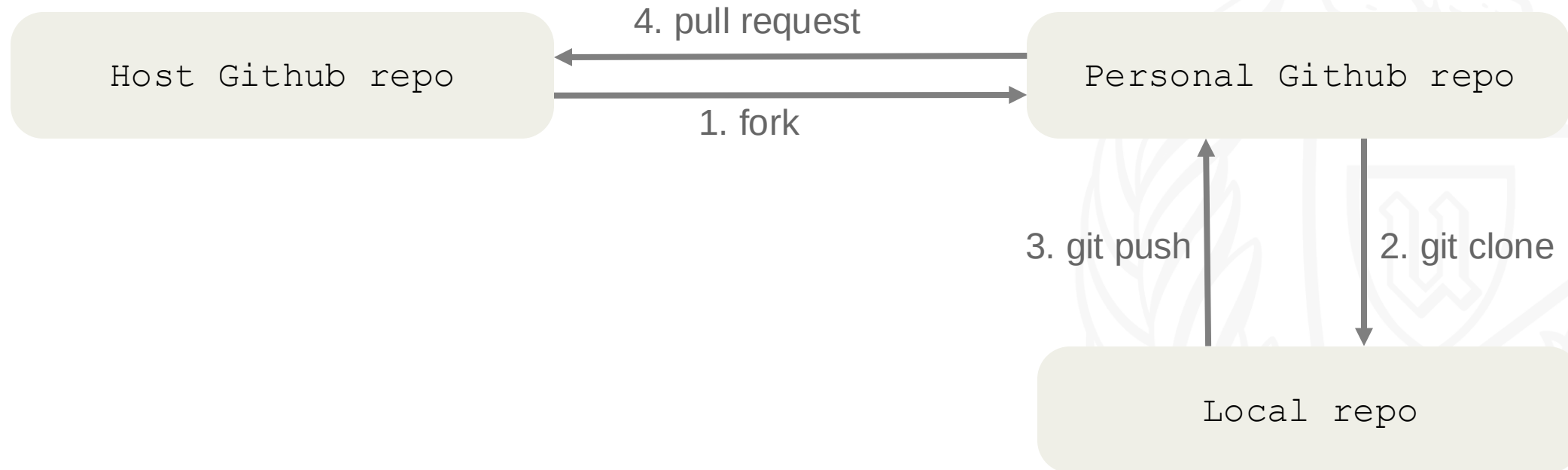
Pushing changes to Github

- `git push <remotename> <branchname>` sends your code in the branch up to the remote
 - Often just `git push`: depends on settings but often equivalent to `git push origin main`

```
$ git push origin main
```

```
(base) jovyan@jupyter-yifancheng-5fub:~/model/git_train$ git push origin main
Username for 'https://github.com': YifanCheng
Password for 'https://YifanCheng@github.com': Copy your personal access token here
```

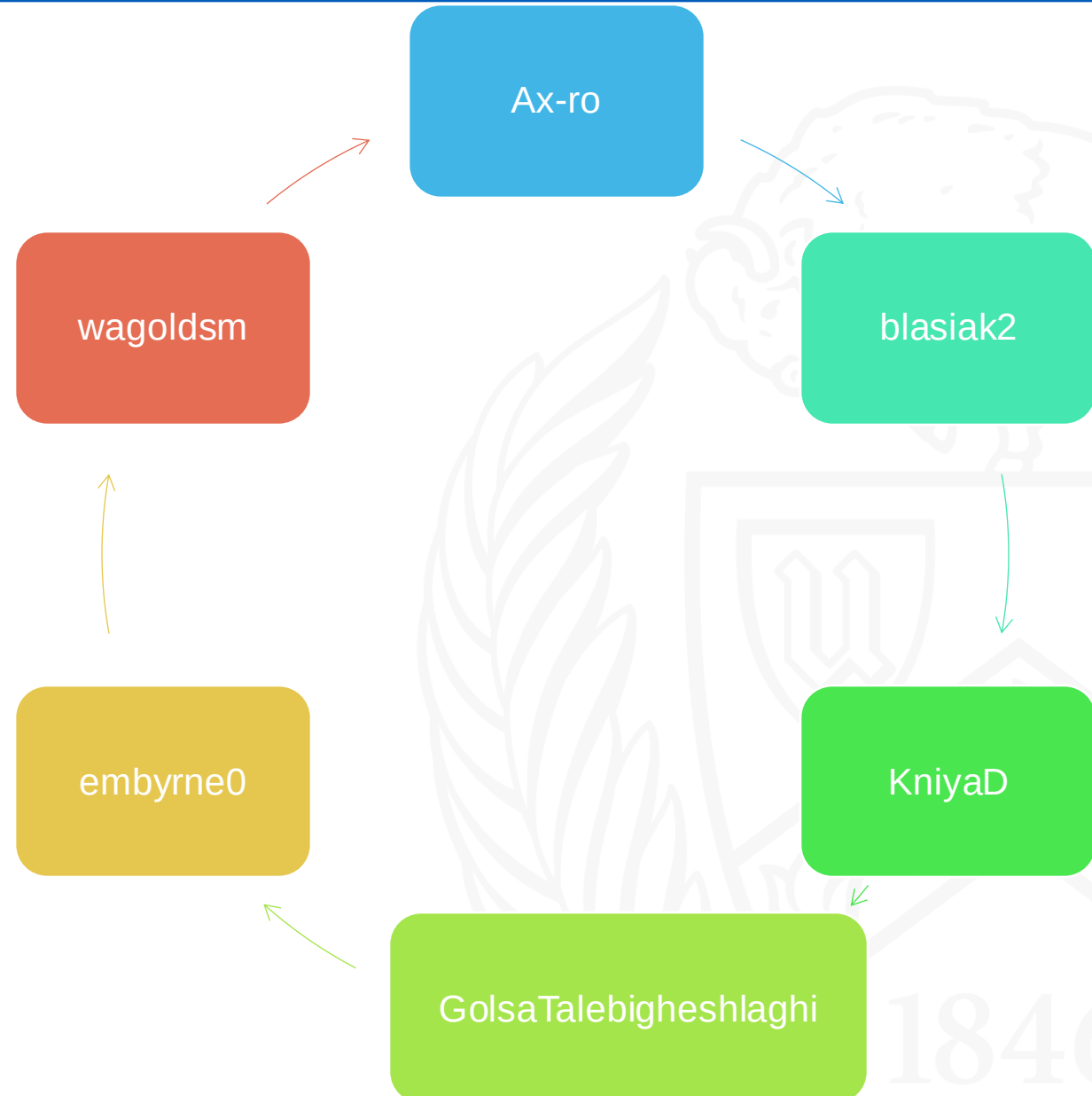
Collaboration workflow



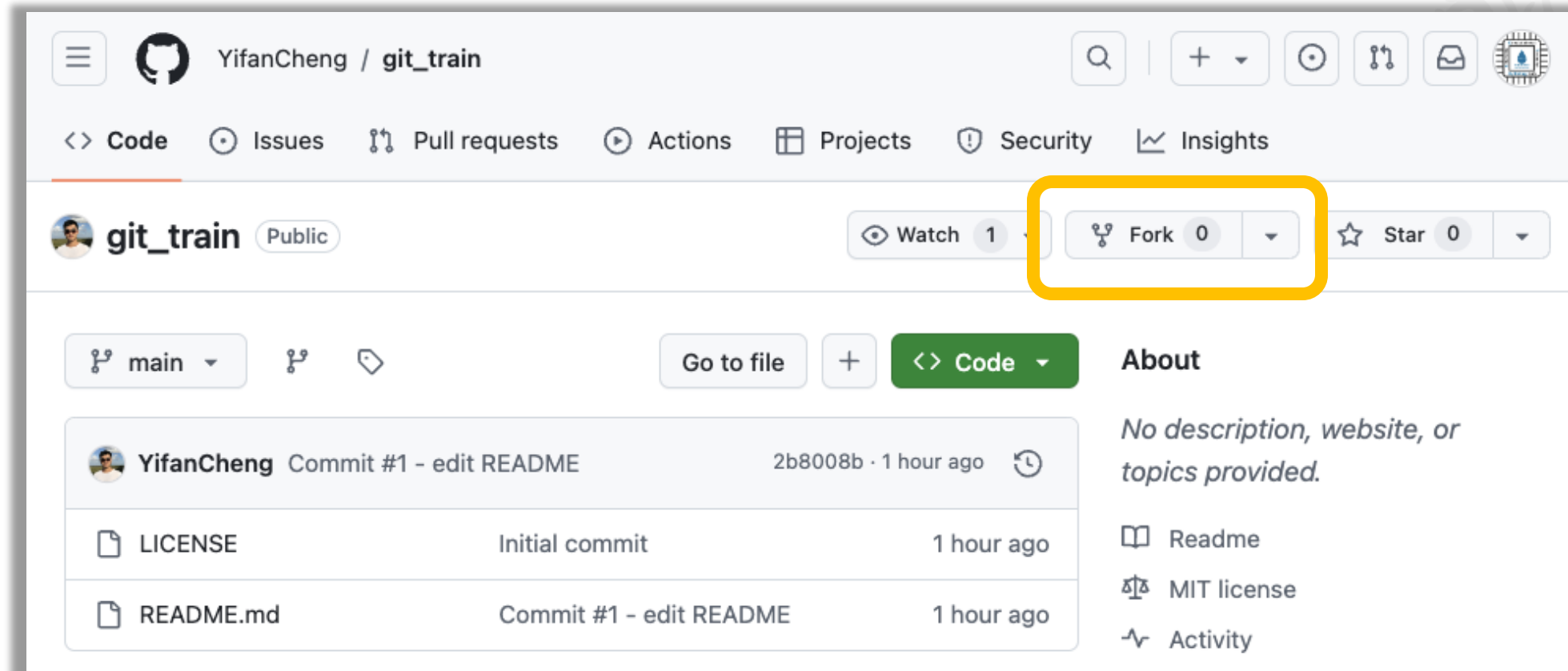
Collaboration

Step 1: **fork** the
“git_train_<initial>” repo
from the person your Github
username pointed to.

wagoldsm will fork **Ax-ro**’s
“git_train_RH” repo



Step 1: Fork



The screenshot shows the GitHub interface for the repository 'git_train' by user 'YifanCheng'. The repository is public. The 'Fork' button is highlighted with a yellow box. Below the repository name, there are buttons for 'main', 'Go to file', and 'Code'. The commit history shows two commits: 'Initial commit' and 'Commit #1 - edit README', both by 'YifanCheng' 1 hour ago. The 'About' section on the right indicates no description, website, or topics are provided, and lists links for 'Readme', 'MIT license', and 'Activity'.

YifanCheng / git_train

Code Issues Pull requests Actions Projects Security Insights

git_train Public

Watch 1 Fork 0 Star 0

main Go to file Code

YifanCheng Commit #1 - edit README 2b8008b · 1 hour ago

LICENSE	Initial commit	1 hour ago
README.md	Commit #1 - edit README	1 hour ago

About

No description, website, or topics provided.

- Readme
- MIT license
- Activity

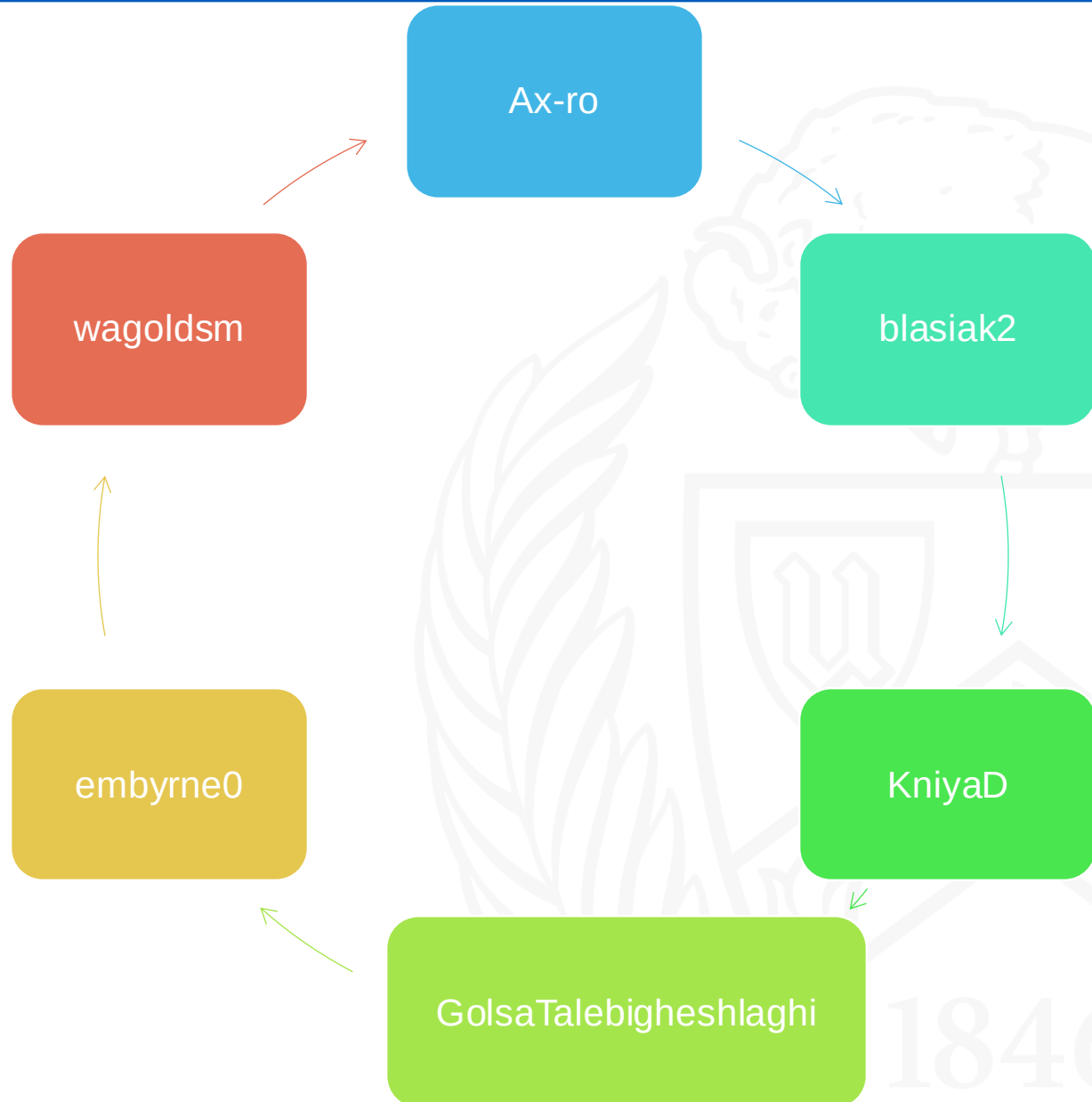
Step 2: Git clone

```
$ cd ~  
$ git clone  
https://github.com/wagoldsm/  
git_train_RH.git  
$ cd git_train_RH  
$ vim README.md
```

Say something nice about this person!

```
$ git commit -a
```

Edit RH's README file

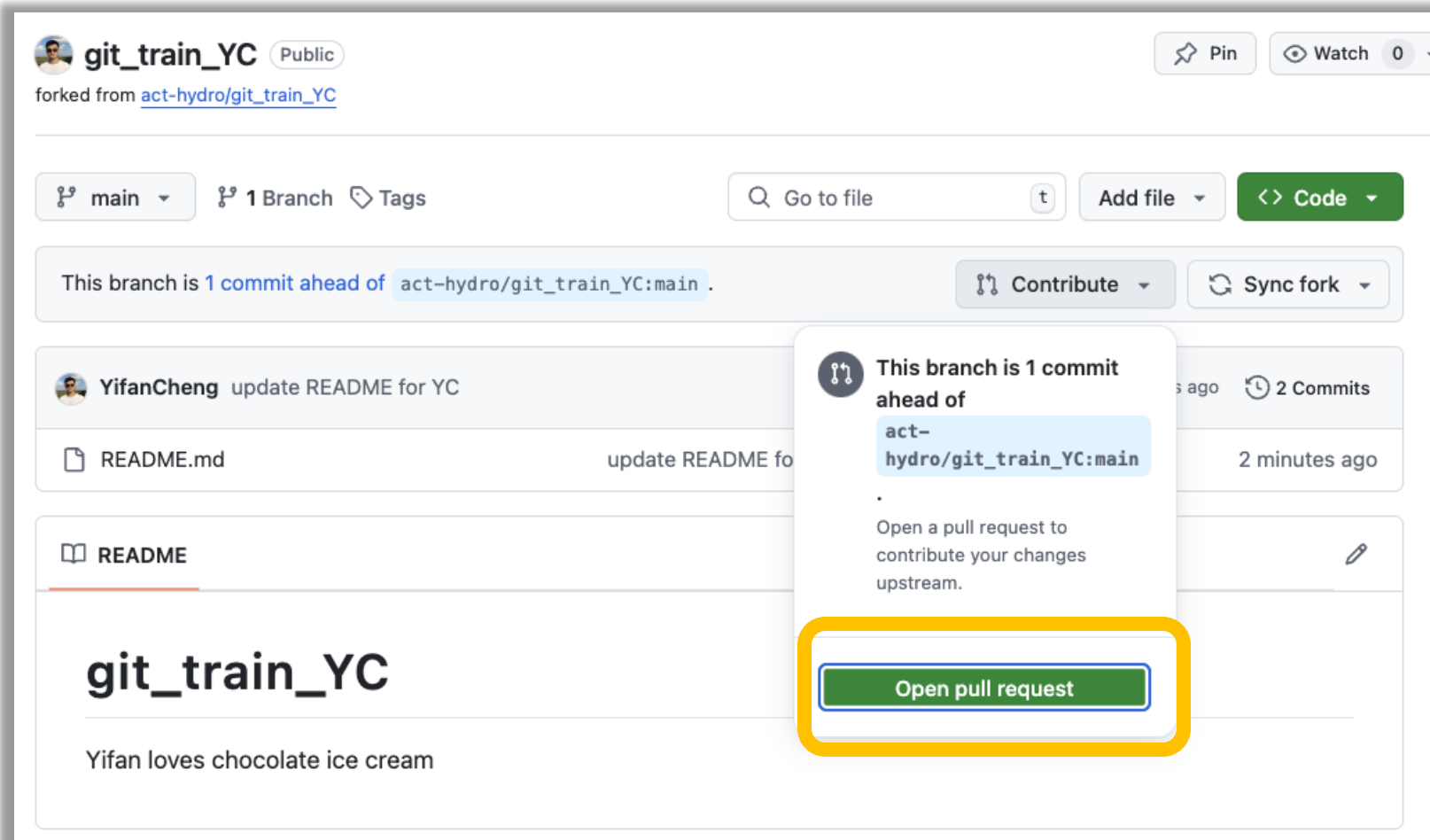


Step 3: Git push

```
$ git push origin main
```

```
(base) jovyan@jupyter-yifancheng-5fub:~/model/git_train$ git push origin main
Username for 'https://github.com': YifanCheng
Password for 'https://YifanCheng@github.com': Copy your personal access token here
```

Step 4: Create pull request



The screenshot shows the GitHub interface for a repository named **git_train_YC**, which is public and forked from [act-hydro/git_train_YC](#). The repository has 1 branch (main) and 0 tags. A notification banner states: "This branch is 1 commit ahead of act-hydro/git_train_YC:main". Below this, a commit by YifanCheng is shown: "update README for YC" (README.md) from 2 minutes ago. A modal dialog is open, stating: "This branch is 1 commit ahead of act-hydro/git_train_YC:main". It includes the instruction: "Open a pull request to contribute your changes upstream." and a prominent green button labeled "Open pull request" which is highlighted with a yellow border.

Step 4: Create pull request

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff comparisons here](#).



base repository: act-hydro/git_train_YC ▾

base: main ▾

⋮

head repository: YifanCheng/git_train_YC ▾

compare: main ▾

✓ **Able to merge.** These branches can be automatically merged.



Add a title

update README for YC

Add a description

Write Preview H B I ≡ <> 🔗 1/2 ≡ ≡ ≡ 📎 @ 🗨️ ⋮

I write something nice in your README file

 Markdown is supported  Paste, drop, or click to add files

☒ Allow edits by maintainers ?

Create pull request ▾